

U2T01: Adapting BERT for NLP tasks

BERT was the first model to make extreme adaptation practical: one pretrained body can be adapted to NLP tasks that would otherwise require enormous labeled datasets and thousands of hours of computing. By today's standards it is not a large model, and yet on classification-style tasks it still performs remarkably well, competitive with models hundreds of times its size.

Adapting BERT comes at different levels of effort, a ladder you climb depending on the task:

Method	What trains	Trainable Parameters
Feature-based adaptation	Nothing in BERT body: its outputs are features that train your model.	0 BERT parameters + your learner parameters
Partial fine-tuning	Head + top encoder layers	Depends on how many layers: approx. 14M with top 2 layers
Full fine-tuning	Everything (BERT + Head)	110M at least

Simple tasks can be achieved with feature-based adaptation; a linear layer (or a classical model) over the last hidden states is enough. But the more complex the task, the more this method struggles, and the more you need fine-tuning. Where exactly that threshold sits is not something you look up: it is something you measure.

In this assignment, your main task is to adapt BERT to four classical NLP tasks, one model per task, and to decide, with your own measurements, which adaptation method to deliver for each one, so you get an empirical understanding of what each rung of the ladder buys and what it costs.

Part 1: Task environment

Adapt BERT to all four tasks below. Each one is a different head architecture on the same body.

- Topic classification: AG News, 4 classes (recommended dataset [fancyzhx/ag_news](#)). One label per text.
- Named Entity Recognition (NER): CoNLL-2003 (recommended dataset [lhoestq/conll2003](#), more docs [here](#)). One label per token.
- Part-of-speech tagging: UD English EWT, 17 tags (recommended dataset [universal-dependencies/universal_dependencies](#), config `en_ewt`). Same per-token shape as NER.

- Extractive QA: SQuAD v1.1 (recommended dataset [rajpurkar/squad](https://rajpurkar.github.io/SQuADv1/)), subsample ~15k training examples. The head predicts start and end positions of the answer span.

You can find and use other datasets for the tasks. Ensure that they have enough data and quality, and please review license terms. If you bring your own datasets, verify they load correctly on recent datasets versions before committing to them.

Subword handling: The tokenizer splits some words into several pieces (playing may become `play + ##ing`), but the datasets carry one label per whole word, so you must decide where the label goes. The convention: the label goes on the first subword of each word, and every other position (continuation subwords, `[CLS]`, `[SEP]`, padding) gets label -100, the special index that CrossEntropyLoss ignores. If labels and tokens don't align, either the loss crashes on mismatched lengths or, worse, it silently trains against misaligned labels and learns garbage. Sanity-check by printing one batch with tokens next to their labels before training. This applies to every token-level task: NER, POS, and QA.

Use bert-base (110M parameters) as the body for your delivered models. You may additionally report DistilBERT (~66M parameters) and BERT-large (~340M parameters) as optional benchmarks; Consider that BERT-large may cause out of memory errors depending on hardware.

No docker compose up environment required for this task. I recommend using Google Colab or Kaggle Notebooks for GPUs. Use HuggingFace Transformers, Tokenizers, Datasets, and Trainer utilities.

Part 2: Experiments

For each task you deliver one model with the adaptation method of your choice, but a delivered model only counts next to a rejected alternative you trained yourself on the same task, with numbers. At least two adaptation methods tested per task, and each of the adaptation methods should appear somewhere in the assignment.

Feature-based adaptation may use different models as consumers for the frozen representations: logistic regression, SVM, random forest, MLP, etc. If you compare two models on top of the frozen representations, that is a valid comparison and counts as your two compared models for that task.

Take care of learning rates. A freshly initialized head wants a big one ($\sim 1e-3$); pretrained encoder layers want a small one ($\sim 2e-5$) — big steps destroy what pretraining built. When a run trains head and body together (partial or full fine-tuning), use two parameter groups with separate learning rates. A single learning rate for everything silently cripples runs, and a crippled run inside a comparison is worse than no run.

One run per configuration is enough. Identical code retrained with a different random seed moves results by $\pm 1-3$ points (head initialization, shuffling, dropout), a method gap smaller than that is noise, not a finding: acknowledge it in the report instead of re-running it.

I recommend generating hypotheses about which methods are best suited for each task, and why and adjusting your hypotheses as you progress.

Part 3: Metrics and evaluations

Identify which metrics and evaluations are adequate for each task; justify why you use each evaluation.

Include metrics for model training according to the methods you use, to help you debug the learning progress: loss values, scores per epoch, seconds per step/epoch, etc.

Your metrics results will be tested against your model and against a teacher's replica of your model to validate your work.

Part 4: Report

Document the experiments' process, provide analysis of the methods used and justify technical decisions, for each model and task. Include a table comparing tasks against methods. Include references for datasets, code and papers used if applicable.

Part 5: Publish

Publish your four final models and their tokenizers to the HuggingFace Hub. Each model should include a model card: training data, metrics, intended use, limitations, proper references. Public or private is your choice. If private, grant access to user [Dexterg83](#).

Deliverables

- Code files: scripts, notebooks, config files, requirements. Use fixed seeds and include documentation about how to replicate the process you used to train your models.
- PDF report.
- HuggingFace Hub repos.

References/Notes

BERT paper (section 5.3 is the feature-based vs fine-tuning comparison this assignment mirrors):

<https://arxiv.org/abs/1810.04805>

Natural Language Processing with Transformers (O'Reilly), chapters 1–3.

Hugging Face docs: fine-tune a pretrained model:

<https://huggingface.co/docs/transformers/training>